



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**INFORME DE LABORATORIO - SISTEMA DE
ALINEACION DE FUTBOL**

Curso: PROGRAMACIÓN II – Sección: B

Docente: Ing. Breno Luque Zuñiga

ALUMNO: Yordan Antonio Cohaila Mamani

Contenido

I. INFORMACIÓN GENERAL	3
Objetivos	3
II. MARCO TEÓRICO	3
Programación Orientada a Objetos (POO).....	3
Clases y Objetos	4
Encapsulamiento	4
Colecciones (List)	4
Windows Forms	5
DataGridView	5
Operaciones CRUD	5
III. PROCEDIMIENTO	6
Paso 1: Creación del proyecto	6
Paso 2: Diseño de la interfaz gráfica	6
Paso 3: Creación de la clase Jugador	7
Paso 4: Creación de la lista de objetos	7
Paso 5: Configuración del ComboBox y DataGridView	7
Paso 6: Programación de las operaciones CRUD	8
Paso 7: Pruebas de funcionamiento	10
IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS	11
¿Qué indican los resultados?	11
¿Qué se encontró?	12
CONCLUSIONES	12
RECOMENDACIONES.....	13
BIBLIOGRAFÍA	13
WEBGRAFÍA	13

I. INFORMACIÓN GENERAL

Objetivos

- Aplicar los conocimientos de Programación Orientada a Objetos (POO) mediante el desarrollo de una aplicación en C# Windows Forms.
- Diseñar una interfaz gráfica que permita registrar y gestionar jugadores de fútbol convocados para un partido.
- Implementar operaciones CRUD (Agregar, Modificar, Eliminar y Mostrar) utilizando listas en memoria.
- Comprender el manejo de eventos y controles de Windows Forms como DataGridView, ComboBox, TextBox y botones.
- Fortalecer la lógica de programación mediante el uso de clases, objetos y colecciones.

Equipos, materiales, programas y recursos utilizados

- Computadora o laptop con sistema operativo Windows.
- Entorno de desarrollo Visual Studio 2022 o superior.
- Lenguaje de programación C#.
- Plataforma Windows Forms.
- .NET Framework/.NET compatible con Windows Forms.
- Recursos bibliográficos y material proporcionado durante el curso.
- Conocimientos de Programación Orientada a Objetos y manejo básico de interfaces gráficas.

II. MARCO TEÓRICO

Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos es un paradigma de programación que permite desarrollar aplicaciones mediante el uso de objetos, los cuales representan elementos del mundo real. Este enfoque facilita la organización del código y permite crear programas más estructurados, reutilizables y fáciles de mantener.

Los principales conceptos utilizados en la POO son clases, objetos, encapsulamiento, herencia y polimorfismo. En esta práctica se aplicaron principalmente clases, objetos y encapsulamiento.

Clases y Objetos

Una clase es una plantilla o modelo que define las características y comportamientos que tendrán los objetos creados a partir de ella.

Un objeto es una instancia de una clase y representa un elemento real con propiedades y características propias.

En el desarrollo de la práctica se creó la clase **Jugador**, la cual contiene atributos como:

- Nombre del jugador
- Posición
- Número de camiseta
- Condición (Titular o Suplente)

Cada jugador registrado dentro del sistema corresponde a un objeto creado a partir de dicha clase.

Encapsulamiento

El encapsulamiento es una característica de la Programación Orientada a Objetos que permite proteger y controlar el acceso a los datos de una clase mediante propiedades y métodos.

En C# esto puede realizarse mediante propiedades get y set, permitiendo acceder y modificar información de forma controlada.

Ejemplo:

```
public string Nombre { get; set; }  
public int Numero { get; set; }
```

Colecciones (List)

Una colección es una estructura que permite almacenar múltiples objetos dentro de una sola variable.

En C#, la colección List<T> permite agregar, modificar y eliminar elementos dinámicamente.

En esta práctica se utilizó una lista de tipo:

```
List<Jugador> listaJugadores = new List<Jugador>();
```

La lista permitió almacenar temporalmente la información de los jugadores registrados sin necesidad de utilizar una base de datos.

Windows Forms

Windows Forms es una plataforma de desarrollo de aplicaciones de escritorio incluida en el entorno .NET, la cual permite crear interfaces gráficas mediante controles visuales.

Esta herramienta facilita la interacción entre el usuario y el sistema mediante componentes como botones, cuadros de texto y tablas.

DataGridView

El control DataGridView permite mostrar información organizada en forma de tabla, facilitando la visualización y manipulación de datos.

En el sistema desarrollado se utilizó para mostrar la lista de jugadores registrados y permitir operaciones como modificar y eliminar registros.

Operaciones CRUD

CRUD es un conjunto de operaciones básicas utilizadas para manipular información dentro de un sistema:

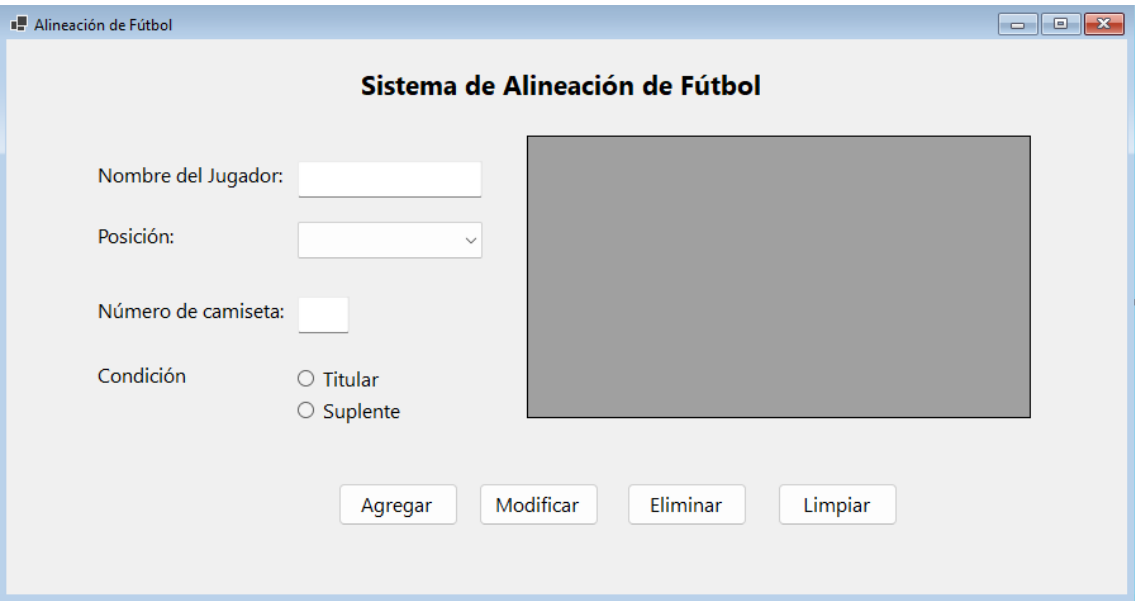
- **Create (Crear):** Agregar nuevos registros.
- **Read (Leer):** Mostrar información registrada.
- **Update (Actualizar):** Modificar información existente.
- **Delete (Eliminar):** Eliminar registros.

Estas operaciones fueron implementadas en la aplicación mediante el uso de botones y listas en memoria.

III. PROCEDIMIENTO

Paso 1: Creación del proyecto

Se creó un nuevo proyecto de tipo **Windows Forms App en C#** utilizando Visual Studio. El proyecto fue desarrollado para gestionar la alineación de jugadores convocados para un partido de fútbol aplicando Programación Orientada a Objetos.



Paso 2: Diseño de la interfaz gráfica

Se diseñó una interfaz gráfica utilizando controles de Windows Forms para permitir el ingreso y visualización de información de los jugadores.

Se agregaron los siguientes componentes:

Componente	Nombre	Función
TextBox	txtNombre	Ingresar nombre
ComboBox	ComboBox	ComboBox
TextBox	txtNumero	Ingresar número
RadioButton	rbTitular	Seleccionar titular
RadioButton	rbSuplente	Seleccionar suplente
Button	btnAgregar	Registrar jugador
Button	btnModificar	Editar jugador
Button	btnEliminar	Eliminar jugador
Button	btnLimpiar	Limpiar campos
DataGridView	dgvJugadores	Mostrar jugadores

Paso 3: Creación de la clase Jugador

Se creó una clase llamada Jugador para representar los datos de cada jugador registrado.

Código utilizado:

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace AlineacionFutbol
6  {
7      6 referencias
8      class Jugador
9      {
10         3 referencias
11         public string Nombre { get; set; }
12         3 referencias
13         public string Posicion { get; set; }
14         3 referencias
15         public int Numero { get; set; }
16         3 referencias
17         public string Condicion { get; set; }
18
19         1 referencia
20         public Jugador(string nombre,
21                         string posicion,
22                         int numero,
23                         string condicion)
24         {
25             Nombre = nombre;
26             Posicion = posicion;
27             Numero = numero;
28             Condicion = condicion;
29         }
30     }
31 }
```

Paso 4: Creación de la lista de objetos

Se creó una colección de tipo List<Jugador> para almacenar la información temporalmente en memoria.

Código utilizado: List<Jugador> listaJugadores = new List<Jugador>();

Esta lista permitió guardar múltiples jugadores sin necesidad de utilizar una base de datos.

Paso 5: Configuración del ComboBox y DataGridView

Se configuró el ComboBox para mostrar las posiciones y el DataGridView para visualizar los registros.

Código utilizado:

```
12 1 referencia
13 private void Form1_Load(object sender, EventArgs e)
14 {
15     // ComboBox
16     cboPosicion.Items.Add("Portero");
17     cboPosicion.Items.Add("Defensa");
18     cboPosicion.Items.Add("Mediocampista");
19     cboPosicion.Items.Add("Delantero");
```

	Nombre	Posición	N° Camiseta	Condición

Paso 6: Programación de las operaciones CRUD

Agregar jugador

Se programó el botón Agregar para crear un objeto y almacenarlo dentro de la lista.

Código:


```

1 referencia
56 private void btnAgregar_Click(object sender, EventArgs e)
57 {
58     string condicion = "";
59
60     if (rbTitular.Checked)
61         condicion = "Titular";
62
63     if (rbSuplente.Checked)
64         condicion = "Suplente";
65
66     Jugador j = new Jugador(
67         txtNombre.Text,
68         cboPosicion.Text,
69         int.Parse(txtNumero.Text),
70         condicion
71     );
72
73     listaJugadores.Add(j);
74
75     MostrarJugadores();
76 }

```

Modificar jugador

Se programó el botón Modificar para actualizar la información seleccionada.

Código:

```

1 referencia
115 private void btnModificar_Click(object sender, EventArgs e)
116 {
117     int fila = dgvJugadores.CurrentRow.Index;
118
119     string condicion = "";
120
121     if (rbTitular.Checked)
122         condicion = "Titular";
123
124     if (rbSuplente.Checked)
125         condicion = "Suplente";
126
127     listaJugadores[fila].Nombre = txtNombre.Text;
128     listaJugadores[fila].Posicion = cboPosicion.Text;
129     listaJugadores[fila].Numero = int.Parse(txtNumero.Text);
130     listaJugadores[fila].Condicion = condicion;
131
132     MostrarJugadores();
133 }

```

Eliminar jugador

Se programó el botón Eliminar para borrar el registro seleccionado.

Código:

```

135     1 referencia
136     private void btnEliminar_Click(object sender, EventArgs e)
137     {
138         int fila = dgvJugadores.CurrentRow.Index;
139         listaJugadores.RemoveAt(fila);
140
141         MostrarJugadores();
142
143         btnLimpiar.PerformClick();
144     }
145

```

Paso 7: Pruebas de funcionamiento

Se realizaron pruebas ingresando diferentes jugadores y verificando las operaciones del sistema.

Resultados obtenidos:

- Se logró registrar jugadores correctamente.

Sistema de Alineación de Fútbol

Nombre del Jugador:

Posición:

Número de camiseta:

Condición ☐ Titular ☒ Suplente

	Nombre	Posición	N° Camiseta	Condición
▶	Messi	Delantero	12	Titular
	Pepe	Portero	33	Suplente

- Se logró modificar información registrada.

Sistema de Alineación de Fútbol

Nombre del Jugador:

Posición:

Número de camiseta:

Condición: ☒ Titular ☐ Suplente

	Nombre	Posición	N° Camiseta	Condición
▶	Messi	Mediocampis...	10	Titular
	Pepe	Portero	33	Suplente

- Se logró eliminar registros.

Sistema de Alineación de Fútbol

Nombre del Jugador:

Posición:

Número de camiseta:

Condición: ☐ Titular ☐ Suplente

	Nombre	Posición	N° Camiseta	Condición
▶	Messi	Mediocampis...	10	Titular

- Los datos se visualizaron correctamente en el DataGridView.

IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

¿Qué indican los resultados?

Los resultados obtenidos muestran que la aplicación desarrollada logró cumplir correctamente con los objetivos planteados para la práctica. Se pudo registrar información de jugadores convocados mediante una interfaz gráfica desarrollada en Windows Forms utilizando Programación Orientada a Objetos.

Asimismo, las operaciones CRUD implementadas permitieron realizar el registro, visualización, modificación y eliminación de jugadores de manera correcta utilizando una lista en memoria (List<Jugador>), sin necesidad de emplear una base de datos.

El control DataGridView permitió visualizar los datos de forma organizada, facilitando la interacción del usuario con el sistema.

¿Qué se encontró?

- Se comprobó que la Programación Orientada a Objetos permite organizar mejor el código mediante el uso de clases y objetos.
- El uso de colecciones como List facilita el almacenamiento y manipulación de datos temporalmente sin depender de una base de datos.
- Se observó que los eventos de Windows Forms permiten ejecutar acciones específicas mediante botones y otros controles.
- Se identificó que algunos errores durante el desarrollo estaban relacionados con configuraciones del diseñador y eventos duplicados, los cuales pudieron resolverse revisando la estructura del formulario.
- Se comprobó que la integración entre la interfaz gráfica y la lógica del programa permite desarrollar aplicaciones funcionales de escritorio de manera sencilla.

CONCLUSIONES

1. Se logró desarrollar una aplicación de escritorio en C# utilizando Windows Forms y Programación Orientada a Objetos para gestionar la alineación de jugadores de fútbol.
2. Se comprobó la importancia del uso de clases, objetos y colecciones (List) para organizar y manipular información de manera estructurada.
3. Las operaciones CRUD implementadas permitieron registrar, visualizar, modificar y eliminar información correctamente, cumpliendo con los objetivos establecidos en la práctica.
4. Se fortalecieron los conocimientos sobre el manejo de eventos y controles de Windows Forms, permitiendo una mejor interacción entre la interfaz gráfica y la lógica del programa.

RECOMENDACIONES

1. Validar los datos ingresados por el usuario antes de almacenarlos para evitar errores durante la ejecución del programa.
2. Implementar mensajes de confirmación al modificar o eliminar registros para mejorar la experiencia del usuario.
3. Mantener una estructura ordenada del código mediante el uso adecuado de clases y métodos para facilitar futuras modificaciones.
4. Como mejora futura, implementar una base de datos para almacenar la información de manera permanente.

BIBLIOGRAFÍA

- Deitel, P., & Deitel, H. (2017). *C# Cómo programar*. Pearson Educación.
- Schildt, H. (2015). *C# 4.0: The Complete Reference*. McGraw-Hill.
- Pressman, R. S. (2014). *Ingeniería del Software: Un enfoque práctico*. McGraw-Hill.

WEBGRAFÍA

- Microsoft. Documentación oficial de C#. Recuperado de: [Microsoft Learn - C# Documentation](#)
- Microsoft. Documentación oficial de Windows Forms. Recuperado de: [Microsoft Learn - Windows Forms](#)